

Task 1: Writing methods

A Calculator model class (MVC) that sums positive integers, throws on negative input, with reset(), add(), getValue() and a temporary Main.

Copilot was used to create 3 mentioned on GitHub assignment ways of method signaturing.

Benefits from Copilot: matched how I'd write this myself for small, conventional methods (guard clause + accumulate); comment-driven generation was a nice way to think through edge cases before writing code; useful for quickly comparing equivalent for/while implementations.

Drawbacks: for code this short writing a whole class to Copilot was as much as just writing a code myself + always checking on what AI has written is just time-consuming in this case.

Task 2: Explaining code

README explanation (with the help of Copilot + checked after)

Copilot was used to explain deeply of how the code is written line by line and additional language checks. I then checked and corrected some, including, for example, why an exception is used instead of silent ignorance of bad input.

Benefits: fast first draft that covers most of the code explanation, lowers the job task for writing a readme by myself.

Drawbacks: explanations are written to explain what the code does instead of why it's designed like that + also possible wrong code explanation may happen, thus it's always crucial to check the AI prompted text.

Overall reflection

Copilot (or any other AI language model) can be used to sufficiently reduce the amount of tasks and job, however it is still very unreliable to post anything AI-prompted without double-checking its output manually (human). Also, writing a small code with AI is as much job as writing by myself. Having a readme draft by Copilot was, however, a huge difference.